



유니티 포트폴리오

길건너 친구들

박 찬 욱

➤ 개요

➤ 작품 소개

→ 길건너 친구들(Crossy Road) 모바일 게임을 모작한 게임

➤ 개발 환경

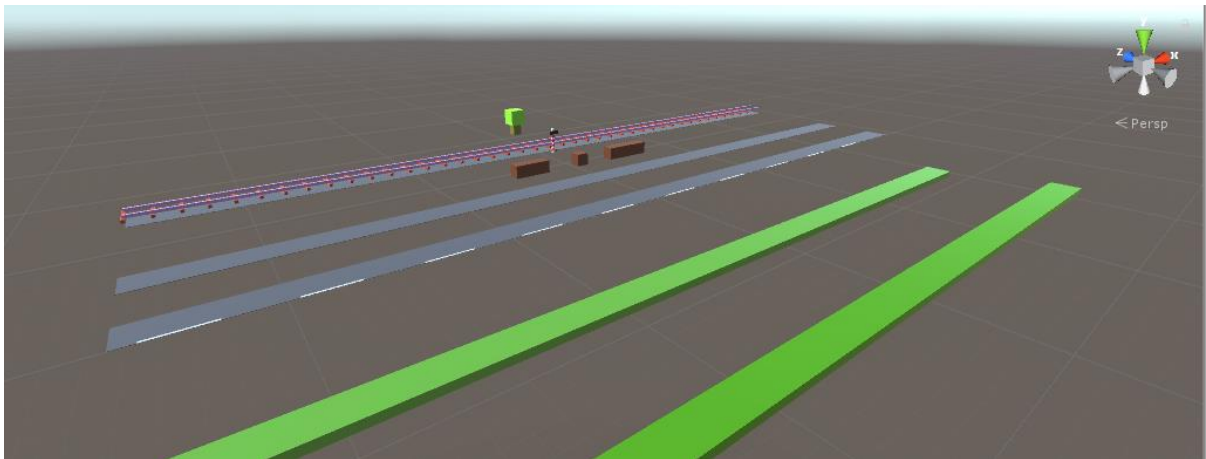
→ Unity 3D

➤ 핵심 기술

➤ 무작위 라인 생성

길건너 친구들의 맵은 어떻게 만들어질까? 원래 게임을 살펴보면 다양한 타일로 구성된 무한한 맵으로 이루어졌다. 하지만 보통은 수많은 지형을 미리 만들어 놓지 않는다. 저 폴리곤 게임이라 할지라도 퍼포먼스가 제 힘을 발휘하지 못할뿐더러 개발 비용적인 측면에서도 효율이 좋지 못하기 때문이다. 이런 점을 감안해보면 비동기적으로 지형을 생성하는 것이 효율적이다.

비동기적이라 함은 플레이어가 앞으로 진행할 때 랜덤으로 타일을 생성하고 이미 사용되어 카메라 범위를 벗어난 타일은 삭제시키는 것이다.



실제 게임에 사용된 지형은 이것이 전부다

이 타일들을 **롱 타일**이라고 지칭했다. 이 롱 타일은 다수의 정사각 타일로 구성되어 있고 플레이어가 한 보 앞으로 전진할 때마다 롱 타일을 맨 끝에 하나씩 생성한다. 물론 처음 게임을 시작할 때는 기본적으로 20칸 정도의 롱 타일을 깔고 시작한다. 이 정도면 한 화면에 전부 들어오고 그 바깥으로 적당한 여유분을 차지하는 타일 개수이다.

```

using UnityEngine;
using System.Collections.Generic;
using System.Linq;

public class StreetGenerator : MonoBehaviour {
    [Singleton]
    public Transform streetsParent;
    public GameObject[] streetPrefabs;
    public GameObject[] streets;

    GameObject farthestStreet;
    GameObject carStreetWithLane;

    public float streetInterval = 10f;
    public float waterGenerationRate = 0.5f;

    // 선이 있는 차도의 생성 비
    public float carStreetWithLaneGenerationRate = 0.5f;

    // 차도에서 차도의 생성 비
    public float carStreetStartGenerationRate = 0.9f;

    public int startStreetCount = 20;

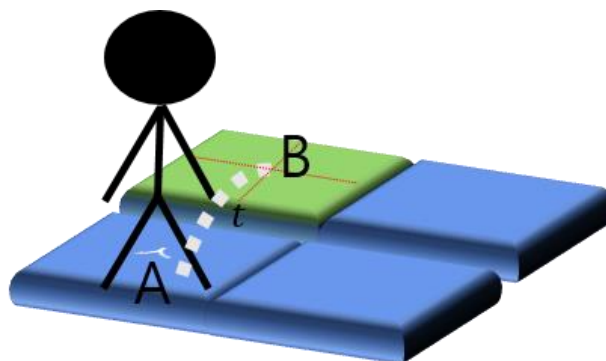
    void Start () {
        public void Refresh() {
            private GameObject GetRandomStreet(List<int> exceptions = null) {

```

StreetGenerator 클래스를 만들었고 Start() 함수에서 startStreetCount만큼 지형을 생성한다. 그리고 플레이어 쪽에서 움직임을 가져갈 때마다 Refresh() 함수를 호출해 맵을 갱신한다. 물론 GetRandomStreet() 함수는 단순히 n분할 시킨 랜덤 로직이 아니라 살짝 더 복잡한 로직이 포함된다. 지형 생성에도 어느정도 일련의 규칙이 존재하기 때문이다. 예를 들면 기차길 타일이 10연속으로 나오게 된다면 플레이어는 앞으로 전진할 수 없다. 이런 점들에 대한 예외처리가 필요하다.

➤ 타일 단위 이동

플레이어가 이동을 취할 때는 복셀 단위가 아니라 **타일 한 칸** 단위로 이동하게 된다.



플레이어는 타일 단위로, 타일 정가운데로 이동한다

이 이동 방식에 자연스러움을 연출하기 위해 **역 선형 보간**을 수행한다. 현재 타일과 이동하려는 타일 두 위치 A, B에 대해 역을 구하면 t 값이 구해지게 된다. 이동을 수행하면 실시간으로 t 값을 증가시키고 플레이어는 그래프상 t 지점의 위치로 지정한다.

길건너 친구들의 이동 메커니즘을 살펴보면 캐릭터가 이동할 때 점프를 하게 되고 방향을 전환

하면 스무스한 회전도 이루어진다.

t 값을 사용해야 하는 이유는 **이동과 동시에 점프, 회전도 진행하기 때문이다**. 즉 이동 타이밍과 점프 타이밍이 맞아 떨어져야 한다. 점프 후 착륙함과 동시에 이동이 끝나 있어야 되기 때문에 시간 값을 공유한다.

점프에 대해 t 값을 180° 로 정규화시킨 **사인 곡선**을 활용하였고, 캐릭터가 방향을 전환할 때는 회전을 하게 되는데 여기 사용된 **구형 선형 보간(Quaternion.Slerp)**에 t 값을 활용하였다.